COSC2673 Computational Machine Learning - Assessment 3

Student: Alissa Nguyen (s4085541)

0. Introduction

This report presents a machine learning workflow for image classification using a modified version of the CRCHistoPhenotypes dataset. The dataset contains small 27 × 27 RGB histology image patches of colon cells. The aim of this project is to train, evaluate, and compare supervised learning models for two related prediction tasks. The first task is to identify whether a cell image is cancerous, using isCancerous as the prediction target. The second task covers classifying the cell type shown in each image, using cellTypeName as the prediction target. Both tasks are challenging because the images are very small, the target classes are imbalanced, and many samples have similar pink and purple staining patterns. This makes the visual differences between classes difficult to distinguish. To make the evaluation more realistic, the dataset is split by patientID rather than by individual image. This ensures that the validation and test sets contain patients who were not seen during training, giving a stricter measure of how well the models generalise to new patients. The workflow includes dataset inspection, exploratory data analysis, patient-level splitting, preprocessing, classical machine learning baseline models, CNN modelling, hyperparameter tuning, data augmentation, threshold tuning, model comparison, final test evaluation, and ethical discussion. The main evaluation metric used throughout the report is macro_f1, as it gives equal importance to each class and is more suitable than accuracy when working with imbalanced classification problems.

Before running the notebook, ensure that:

1. A Python environment is installed, such as Anaconda or standard Python with Jupyter Notebook or JupyterLab.
2. The data folder is placed in the same directory as the notebook.
3. The data folder contains images/, data_labels_mainData.csv, and data_labels_extraData.csv.
4. The required Python libraries are installed: pandas, numpy, matplotlib, Pillow, scikit-learn, torch, and torchvision.

To run the notebook:

5. Start Jupyter Notebook or JupyterLab.
6. Open a3_s4085541.ipynb.
7. Restart the kernel.
8. Run all cells from top to bottom in order.

The notebook loads the image labels and image files, prepares the two classification tasks, creates patient-level training, validation, and test splits, preprocesses the data, trains and evaluates several models, applies advanced techniques, selects the final models, and evaluates them on held-out unseen-patient test sets. The final results are shown in the notebook through metric tables, classification reports, confusion matrices, and training and validation plots.

A. Task Definition

This assignment involves two supervised image classification tasks using colon cell histology images. Each input is a small RGB image patch with the shape 27 × 27 × 3. Both tasks use the same image format, but they predict different target variables.

	1. Cancer Classification	2. Cell-Type Classification
Task	binary image classification	multi-class image classification
Input	a 27 × 27 × 3 RGB histology image	a 27 × 27 × 3 RGB histology image
Target variable	`isCancerous`	`cellTypeName`
Classes	0 for non-cancerous and 1 for cancerous.	epithelial, fibroblast, inflammatory, and others
Learning experience	data_labels_mainData.csv, data_labels_extraData.csv	data_labels_mainData.csv
Evaluation focus	model performance on unseen patients.	model performance across all cell-type classes on unseen patients

Table 1. Summarises the two prediction tasks used in this report.

The main evaluation metric used is macro_f1. This metric is more suitable than accuracy because both classification tasks have imbalanced classes. Accuracy can be inaccurate when a model performs well on the majority class but poorly on the smaller classes. On the other hand, macro_f1 gives equal weight to each class, so it provides a clearer view of whether the model performs consistently across all categories. Another objective is to assess how well the models generalise to unseen patients. To support this, the dataset is split using patientID instead of randomly splitting individual images. This makes sure that images from the same patient do not appear across the training, validation, and test sets. Although this makes the evaluation stricter, it gives a more realistic estimate of how the final models may perform when applied to data from new patients.

B. Dataset Description and EDA

B.1 Dataset overview

There are image patches and two label datasets in the dataset. Each image patch has the size 27 × 27 pixels and three colour channels. Label data is stored in data_labels_mainData.csv and data_labels_extraData.csv datasets. Labels for both prediction tasks can be found in the first one, while labels for the second prediction task are located in the second file. To simplify the cancer classification task, all the labels from both label files were merged into one. Since only the first file contains the cellTypeName variable, it was used for the cell-type classification task. After matching the image files with their labels, the cancer classification task contained 20 280 images from 98 patients, whereas the cell-type classification task included 9 896 images from 60 patients. Both prediction tasks are class-imbalanced: the number of cancerous images in the former is 7 069, while the number of non-cancerous images is 13 211; the number of images in cell-type classes are as follows: epithelial 4 079, inflammatory 2 543, fibroblast 1 888, others 1 386.

```
#final target distribution checks after image-path filtering
print("Final cancer task target distribution:")
display(cancer_df["isCancerous"].value_counts(dropna=False))

print("\nFinal cell-type task target distribution:")
display(celltype_df["cellTypeName"].value_counts(dropna=False))
```

```
Final cancer task target distribution:
isCancerous
0    13211
1     7069
Name: count, dtype: int64

Final cell-type task target distribution:
cellTypeName
epithelial      4079
inflammatory    2543
fibroblast      1888
others          1386
Name: count, dtype: int64
```

Figure B.1 Dataset overview and target distributions for the two classification tasks

The main challenges with the given dataset include: 1) class imbalance, 2) image patches have low resolution, 3) image patches come from different patients, and 4) there are more images per some patientIDs than for others.

B.2 Class distribution analysis

Both prediction tasks have an imbalanced distribution (see Figure B.2). In particular, the first prediction task includes more images with label 'isCancerous = 0', while in the second one the class epithelial has more entries compared to the other three classes.

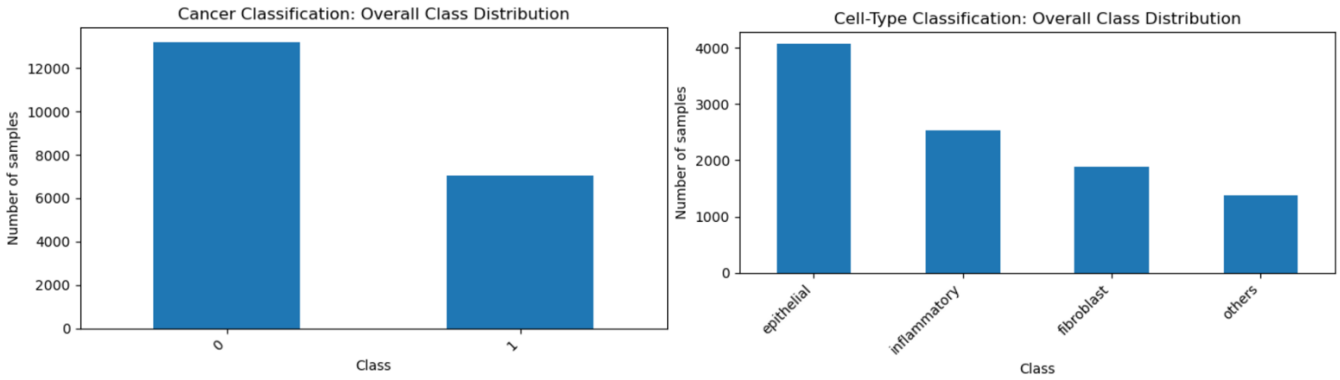

Figure B.2. Class distribution for cancer classification and cell-type classification

In terms of model performance, this class imbalance indicates that accuracy is an unreliable metric of performance because even poorly trained models can have a reasonably high accuracy due to class dominance. To address this problem, macro_f1 score was chosen as the main metric of evaluation in this assignment because unlike accuracy, it takes the performance of all the classes into account. In addition, class imbalance calls for the necessity of building confusion matrices since a single number of overall accuracies is insufficient to identify errors in the predictions.

B.3 Visual characteristics of the images

Some sample images from both classes are depicted below in Fig B.3. The images have the size 27 × 27 pixels, while the staining pattern appears to be quite similar for different classes. There can be some cell structures present in some images.

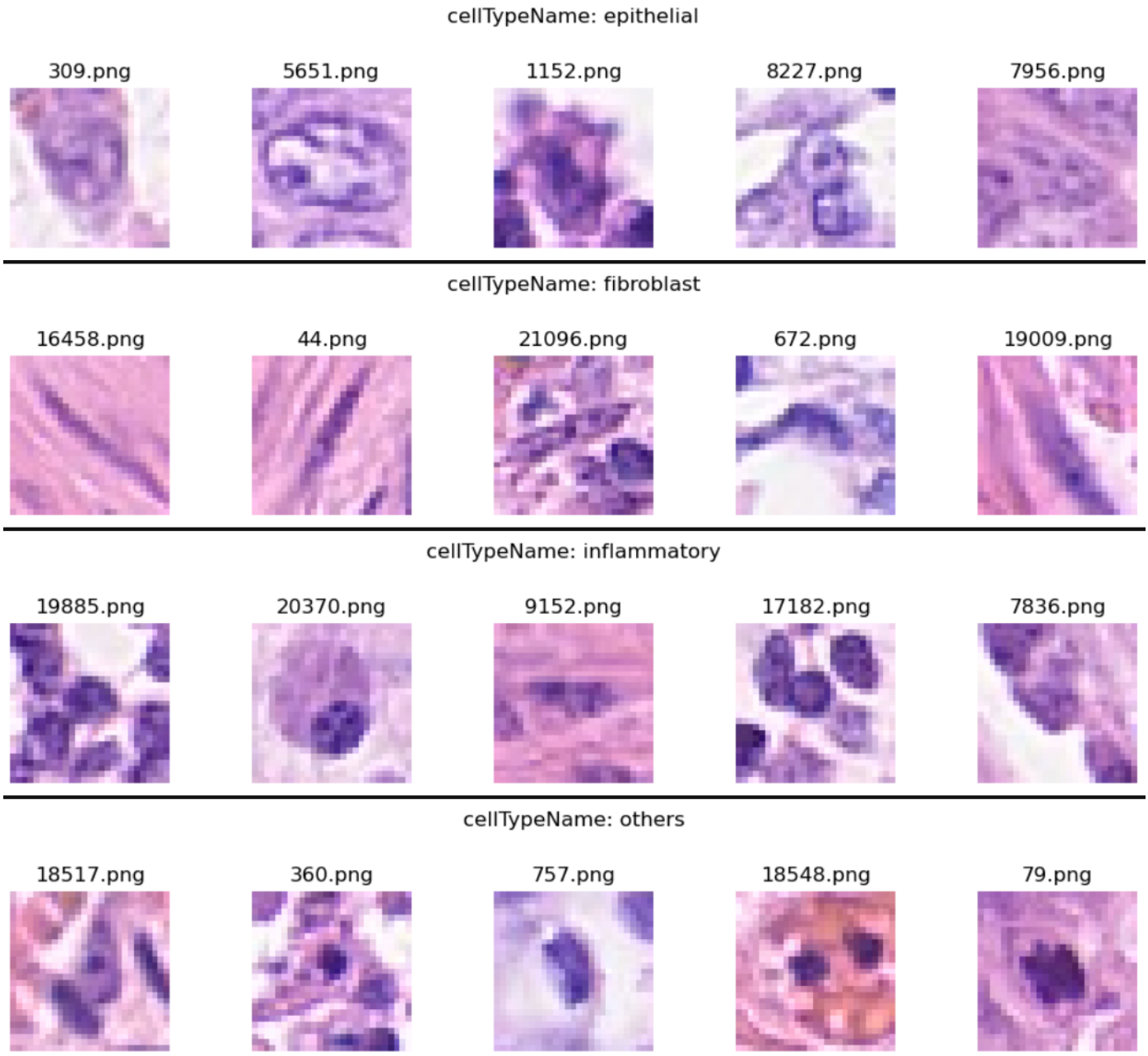

Figure B.3. Example 27 × 27 RGB histology image patches by class.

The visual analysis of the given image dataset demonstrates that the classification task is rather challenging since no clear patterns can be detected in terms of object presence and colour rules. Thus, the model needs to learn smaller-scale features like stain patterns, textures, and even local cell structure. As a result, it seems that the assignment can benefit from using CNN architecture which is specifically developed for such tasks.

B.4 Patient-level image distribution

The given dataset is class-balanced in terms of images but patient-imbalanced in terms of the number of images. This information is important for designing an evaluation framework since models should be evaluated on their ability to generalize to unknown patients, not only unknown images. Indeed, the random image-based splitting can produce images belonging to the same patient in the train and validation datasets, hence causing patient leakage. This can lead to overfitting, i.e. models will learn features specific to patients rather than to a wide range of patients. To address this issue, the patient-based splitting was done in the data preprocessing part of this assignment. The primary idea of using patient-based splitting in this assignment lies in the need to build separate patient-based train, validation, and test datasets. As a result, models will generalize to patient-level, as they should.

B.5 Unsupervised EDA

Unsupervised EDA was performed in the current assignment to test if image pixels are separated naturally into different classes without using supervised learning models. Specifically, flattened image features were analysed with PCA, while K-Means was run on the dataset as well.

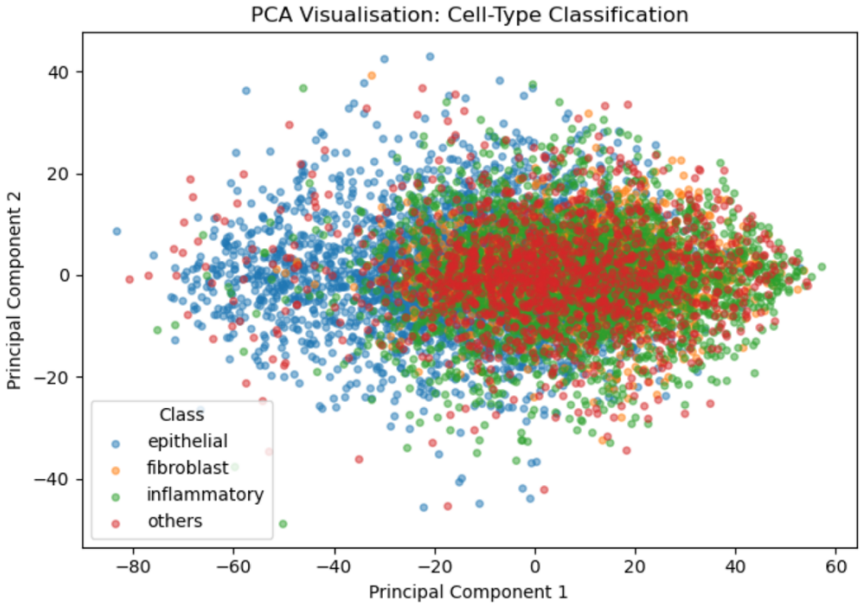

Figure 3. PCA visualisation of flattened image features.

As shown, classes are quite overlapped in PCA projections. This result suggests that the primary linear features of the pixels cannot be effectively used for class separation, hence supporting the idea that the given problem cannot be solved with simple models. This conclusion is supported by K-Means results; clusters created with this method are not fully consistent with actual classes. Overall, the above analysis demonstrates that supervised learning is required for the task, as well as models that can effectively learn features in image data.

C. Data Pre-processing

The preprocessing phase ensured that image files and label files were ready for modelling. This involved assessing data quality, creating task-specific datasets, patient-level train-validation-test splitting, target encoding, normalising image values, flattening for classical models, and preparing image tensors for CNN models.

C.1 Data cleaning and task-specific datasets

First, data quality was evaluated by checking the contents of the label files. This included checking the shape, column names, missing values, duplicates, distribution of the target variables, number of unique patients, and whether each row in label files has a corresponding image file in the images folder. Missing values were not found for the relevant label columns in any label file, and there were no duplicate rows in the main or extra label files. Each row of each label data set had a matching image file in the images folder. This was important since mismatch between label rows and image files would be detrimental to modelling process. Two task-specific data sets were then created. cancer_df was obtained by merging data_labels_mainData.csv and data_labels_extraData.csv, since both label files have the target isCancerous. On the other hand, celltype_df uses only data_labels_mainData.csv, since cellTypeName is available only in this file. This split will help avoid using invalid rows in modelling. Furthermore, keeping separate data sets for each modelling task will help maintain clarity since different tasks involve different datasets, target variables, splitting procedure, pre-processing steps, and evaluation.

C.2 Patient-level train, validation, and test splitting

One of the most important preprocessing steps was to split data by patientID. This is because we need to ensure that all images belonging to the same patient will always end up in the same split (training, validation, or test). Otherwise, it will be possible for patients in the training split to also be present in validation and/or test splits. This is called patient leakage, and it can adversely impact the final results, causing model overfitting. In this case, the model would learn patient-specific properties (staining patterns, etc.), leading to poor results when evaluated on unseen patients. The cancer classification task was split into 12 319 training samples from 62 patients, 4 005 validation samples from 16 patients, and 3 956 test samples from 20 patients. The cell-type classification task was split into 6 215 training samples from 38 patients, 1 842 validation samples from 10 patients, and 1 839 test samples from 12 patients.

```
cancer_train_df, cancer_val_df, cancer_test_df = patient_level_split(
    cancer_df,
    patient_col="patientID",
    test_size=0.20,
    val_size=0.20,
    random_state=RANDOM_STATE
)

print("Cancer classification split sizes:")
print("Training samples:", len(cancer_train_df))
print("Validation samples:", len(cancer_val_df))
print("Test samples:", len(cancer_test_df))

print("\nCancer classification patient counts:")
print("Training patients:", cancer_train_df["patientID"].nunique())
print("Validation patients:", cancer_val_df["patientID"].nunique())
print("Test patients:", cancer_test_df["patientID"].nunique())
```

```
Cancer classification split sizes:
Training samples: 12319
Validation samples: 4005
Test samples: 3956

Cancer classification patient counts:
Training patients: 62
Validation patients: 16
Test patients: 20
```

```
celltype_train_df, celltype_val_df, celltype_test_df = patient_level_split(
    celltype_df,
    patient_col="patientID",
    test_size=0.20,
    val_size=0.20,
    random_state=RANDOM_STATE
)

print("Cell-type classification split sizes:")
print("Training samples:", len(celltype_train_df))
print("Validation samples:", len(celltype_val_df))
print("Test samples:", len(celltype_test_df))

print("\nCell-type classification patient counts:")
print("Training patients:", celltype_train_df["patientID"].nunique())
print("Validation patients:", celltype_val_df["patientID"].nunique())
print("Test patients:", celltype_test_df["patientID"].nunique())
```

```
Cell-type classification split sizes:
Training samples: 6215
Validation samples: 1842
Test samples: 1839

Cell-type classification patient counts:
Training patients: 38
Validation patients: 10
Test patients: 12
```

Figure C2. Patient-level train, validation, and test split summary

After splitting, a leakage check was performed to ensure that no patient was repeated across multiple splits. Overlaps between training-validation pairs, training-test pairs, and validation-test pairs of patients were 0.

```
print("Cell-type classification leakage check:")
check_patient_leakage(
    celltype_train_df,
    celltype_val_df,
    celltype_test_df,
    patient_col="patientID"
)
```

```
Cell-type classification leakage check:
Train/Validation patient overlap: 0
Train/Test patient overlap: 0
Validation/Test patient overlap: 0
No patient leakage detected.
```

Figure C2.2 Patient leakage checks for train, validation, and test sets

While patient leakage check guarantees better results, class distribution can be uneven among the splits. This can be detrimental to model performance and training. However, avoiding patient leakage is more important than having identical class distribution among the three splits.

C.3 Image loading and normalisation

Each image file is read and represented in the RGB format. Images have the size 27 × 27 × 3. Pixel values are represented by integers in the range 0-255. This is changed to the 0-1 range to normalise images, and this step will be discussed further in the subsequent section. The output of image pre-processing has two forms: flattened or tensorized. Flattened images are used for classical machine learning models. Images are flattened into vectors of shape (27 * 27 * 3,). Thus, each flattened image has 2,187 elements. The CNN model receives the tensorized version of the images, which is a torch. Tensor object of shape (3, 27, 27).

C.4 Target encoding

Target encoding was done separately for each of the two tasks, depending on how the targets were represented. The variable isCancerous is already represented by numbers (0 and 1). Therefore, no label encoding needs to be done for this target variable. The target variable cellTypeName represents the class name as a string. For example, epithelial, fibroblast, inflammatory, and others are examples of class names. This information should be encoded as numbers, which are passed to scikit-learn models and PyTorch loss function. Thus, LabelEncoder was used to encode the class strings.

C.5 Scaling and model-specific preparation

The pre-processing steps for classical models (Logistic Regression, SVM RBF, and Random Forest Classifier) and CNNs differ. First, StandardScaler was used in a pipeline for LogisticRegression and SVM RBF. They are susceptible to scale since they depend on optimization, distances, and margins. Thus, scaling of the flattened image pixel vectors is expected to improve model performance. RandomForestClassifier, which is based on decision-tree splits, does not depend on pixel-scale, although it still uses flattened pixel vectors. In the case of the CNN, StandardScaler is not used. The normalised image tensors are used instead of scaled images.

C.6 PyTorch datasets and dataloaders

For CNN modelling, custom PyTorch datasets and data loaders are created. They store image tensors along with their labels in an easily accessible way. Dataloaders allow the use of batch-size for training the CNN. Shuffle operation is performed on the training dataloaders to introduce randomness in CNN training. Validation and test data loaders do not need shuffle operation, since no randomness is required for evaluation. Additionally, class weights are prepared for CNN experiments where class balance should be considered.

D. Model Development and Performance Analysis

This section examines the performance of various supervised models that were used to solve the histology classification problems described in section C. The goal was not just to determine the best-performing classifier, but to see how well different model types worked for a relatively simple image recognition task.

Four model families were compared:

1. Logistic Regression
2. SVM RBF
3. Random Forest Classifier
4. Improved CNN

The same model families were utilised for both the cancer classification task and the cell-type classification task. This ensured the consistency of the comparison and provided a controlled setting, comparing the linear base model, non-linear classical models, and a custom image-based model. Invalidation, five metrics were used to measure performance: accuracy, macro_f1, weighted_f1, macro_precision, and macro_recall. The main metric was chosen to be macro_f1, since both classification tasks suffer from imbalance and require decent performance on every class, not just on a majority class.

D.1 Logistic Regression

Logistic Regression was chosen as a baseline for a supervised classifier. Although it includes “regression” in its name, this term refers to a different type of machine learning task, and this model serves for classification purposes in this paper. It creates a linear decision boundary between the classes based on linear features. For this task, images were flattened into one-dimensional pixel vectors consisting of 2,187 numerical features each. StandardScaler was applied in the pre-processing pipeline since logistic regression relies on numeric features. The strength of Logistic Regression is that it is a simple, fast model that can be used as a baseline. With it, the question of whether the classes can be distinguished based on a linear combination of features can be answered. The main drawback of Logistic Regression is that it cannot inherently process image information due to flattening of the original image patches. Once flattened, a model sees all the neighbouring pixels independently and does not have the ability to consider any local image structure. This can be seen from the poor results of logistic regression for this dataset – its cancer validation macro_f1 is approximately 0.7710 and the cell-type macro_f1 is approximately 0.4324.

D.2 SVM with RBF Kernel

SVM with a radial basis function kernel (SVM RBF) was used as a non-linear classical model. A support vector machine tries to create a maximum-margin decision boundary between classes. The use of the RBF kernel allows a support vector machine to form complex decision boundaries. This classifier was chosen for this dataset since the previous analysis indicated that there is some non-linearity in the classes’ separation. Like logistic regression, SVM RBF relied on flattened image vectors. StandardScaler was applied in pre-processing. The key advantage of SVM RBF is that it can create more complex decision boundaries than Logistic Regression. The limitation of the model is that it does not work with image structure, and it is relatively heavy compared to logistic regression. From the validation metrics, it can be seen that the SVM RBF classifier outperformed Logistic Regression in terms of accuracy. Particularly, it produced the cancer validation macro_f1 equal to approximately 0.8568 versus 0.7710 for the former. However, in the case of the cell-type classification, SVM RBF only produced a validation macro_f1 of approximately 0.5053, underperforming compared to the improved CNN.

D.3 Random Forest Classifier

As a non-linear ensemble classifier, Random Forest Classifier was chosen to complement SVMs. Multiple decision trees are trained and combined into an ensemble model. This allows to create complex decision boundaries and decreases model variance by combining results of multiple classifiers. This classifier was chosen for this dataset because it provides an alternative non-linear baseline to SVM RBF. Random Forests are not reliant on StandardScaler for the training and do not require feature pre-scaling. However, similarly to the previous models, Random Forest was trained on flattened image vectors and, thus, does not utilise image structures. The Random Forest produced cancer validation macro_f1 of approximately 0.8277, outperforming Logistic Regression, but not SVM RBF or the improved CNN. Its cell-type validation macro_f1 was approximately 0.4345, which indicates that even though some non-linear pixel interactions were captured by the model, flattened images may not be the most suitable representation for such complex multi-class tasks.

D.4 Improved Convolutional Neural Network

An improved CNN was developed as an image-based model. Contrary to the previous three, it was trained directly on image tensor inputs with shapes of 3 × 27 × 27, meaning that image patches retained their spatial structure. Convolutional, batch norm, ReLU, max pool, dropout, and fully connected layers made up the CNN structure. It is expected that such a structure will provide superior performance in image processing since it retains the structure and captures local image features. Based on the EDA results, CNNs should work well in the current classification context because the task depends on the colour, texture, and local image structure. Invalidation, this model provided the highest initial performance. It produced cancer validation macro_f1 of approximately 0.8764 and cell-type validation macro_f1 of approximately 0.6380. Thus, the CNN achieved better initial results, however, it only satisfied the minimum threshold of the cell-type classification task.

	task	model	accuracy	macro_f1	weighted_f1	macro_precision	macro_recall	best_epoch	best_val_macro_f1
6	Cancer Classification	Improved CNN	0.879650	0.876364	0.879439	0.877790	0.875145	5.0	0.876364
2	Cancer Classification	SVM RBF	0.859925	0.856775	0.860015	0.856332	0.857250	NaN	NaN
4	Cancer Classification	Random Forest	0.836954	0.827703	0.833794	0.846749	0.820706	NaN	NaN
0	Cancer Classification	Logistic Regression	0.773034	0.770972	0.774287	0.770570	0.776634	NaN	NaN
7	Cell-Type Classification	Improved CNN	0.686754	0.637979	0.694149	0.633793	0.651460	11.0	0.637979
3	Cell-Type Classification	SVM RBF	0.602606	0.505300	0.587570	0.566720	0.550925	NaN	NaN
5	Cell-Type Classification	Random Forest	0.574376	0.434521	0.515536	0.586220	0.516340	NaN	NaN
1	Cell-Type Classification	Logistic Regression	0.528773	0.432366	0.525408	0.441512	0.453209	NaN	NaN

Figure D.4 Validation performance comparison for the supervised models.

It can be seen that a CNN outperformed other models in both tasks, thus validating the intuition about the importance of image structure. However, a cancer CNN model needed improvement in order to satisfy the assessment criterion of at least 0.90 validation score. This fact motivated the subsequent section.

D.5 Neural Network Training Behaviour

To analyse the behaviour of the CNNs across training epochs, training and validation metrics were plotted.

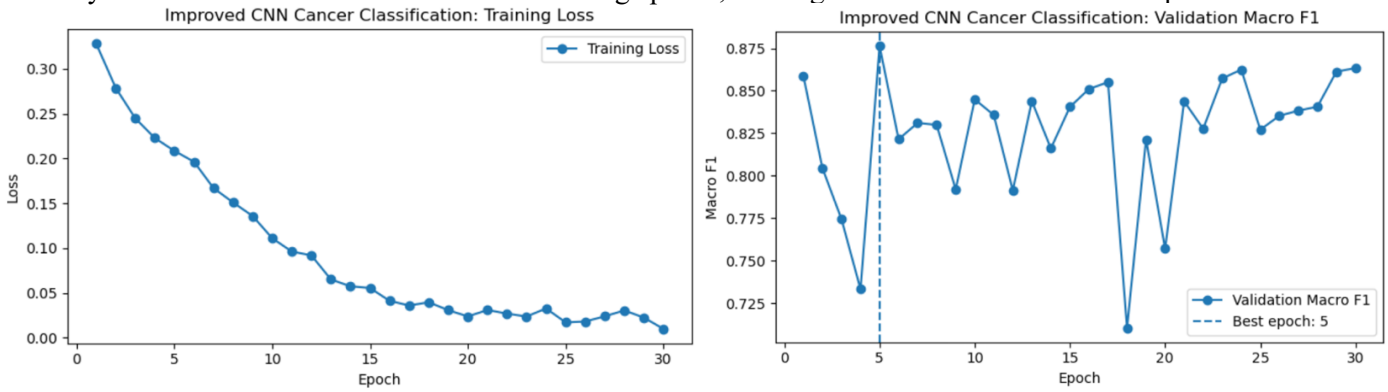

Figure D.5 Training loss and validation macro F1 for the cancer Improved CNN.

From this plot, it can be seen that although training loss decreased consistently, validation macro_f1 fluctuated in epochs. This is important since achieving low training loss does not necessarily imply good generalisation performance.

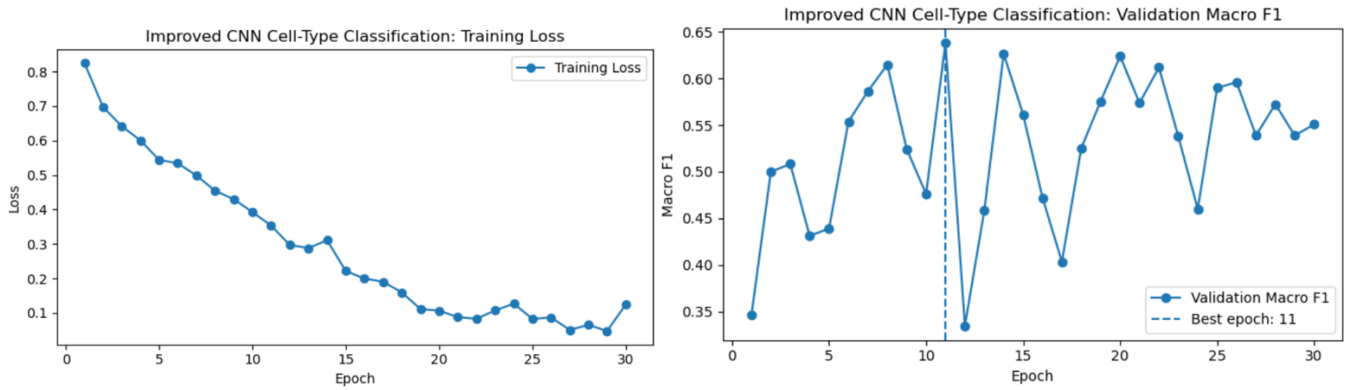

Figure D.5.2 Training loss and validation macro F1 for the cell-type Improved CNN.

In this case, the CNN also exceeded the threshold requirement, but it produced lower macro_f1 performance compared to the cancer CNN. This is expected since cell-type classification requires distinguishing among four classes, and it is more difficult to separate visually similar objects. Also, from the plots above, it can be observed that the validation

metrics were unstable throughout the epoch iterations. Thus, it becomes clear that selecting the best model cannot be done based solely on the training loss. A neural network can keep decreasing the latter while providing almost zero performance gain on the validation set. Therefore, in this particular case, the improved CNN was selected based on validation performance and not just the training loss value.

D.6 Controlled Hyperparameter Experiments

The following experiment was conducted for the cancer CNN since it was the strongest among all classifiers but still failed to meet the 0.90 macro_f1 criterion. Learning rate, weight decay, and class-weighted loss were altered to improve generalisation performance. The reduction of learning rate was expected to increase validation stability since smaller updates would reduce the chance of missing the global minimum of the decision boundary. The usage of weight decay was motivated by potential overfitting. The third modification to the training procedure was introducing class-weighted loss, which was expected to improve imbalanced classification tasks. However, the tuning results showed that changing the mentioned hyperparameters did not lead to additional gains in model performance. The best performing model produced cancer validation macro_f1 of approximately 0.8727. Although this was better than the baseline classifier's macro_f1 of approximately 0.8568, it was inferior to the improved CNN of approximately 0.8764. It is interesting to note that this experiment indicated the importance of improving model generalisation rather than training itself. This motivated further investigation into data augmentation and threshold tuning.

D.7 Summary of Model Behaviour

Overall, it can be said that a pattern is noticeable in the behaviour of the models considered above. A linear classifier was useful, but not sufficient. Non-linear classifiers were better in terms of accuracy; however, they suffered from flattened image representations. CNNs were best performing overall since they took full advantage of image structures. But again, model complexity alone was not enough. Only the improved CNN was good enough after further improvement.

E. Advanced Techniques

The advanced techniques stage focused on the cancer classification task. In the earlier model comparison, the Improved CNN was the strongest model family, but its validation macro_f1 was 0.8764, which was still below the required 0.90 target. The cell-type CNN had already exceeded its required 0.60 target, so further improvement was prioritised for the cancer model.

Three improvement strategies were tested in sequence:

1. Hyperparameter tuning
2. Data augmentation
3. Decision threshold tuning

This order was used to first determine whether optimiser level changes were sufficient before applying techniques aimed at improving generalisation and adjusting the final decision rule.

E.1 Hyperparameter tuning

The first improvement attempt was hyperparameter tuning for the cancer CNN. The tuning experiments tested learning rate, weight decay, and class-weighted loss because these settings directly affect neural network training. The learning rate controls the size of the optimiser updates during training. A smaller learning rate can make training more gradual and stable, although it may also slow the learning process. Weight decay was tested as a regularisation method to reduce overfitting by penalising overly large model weights. Class-weighted loss was tested because the cancer classification task has imbalanced classes, with fewer cancerous images than non-cancerous images.

	task	model	accuracy	macro_f1	weighted_f1	macro_precision	macro_recall	best_epoch	best_val_macro_f1	learning_rate	weight_decay	use_
1	Cancer Classification	CNN lr=0.0003 class weights	0.875406	0.872699	0.875531	0.871992	0.873490	9	0.872699	0.0003	0.0001	
2	Cancer Classification	CNN lr=0.0001 no class weights	0.872659	0.870073	0.872870	0.868970	0.871419	6	0.870073	0.0001	0.0001	
0	Cancer Classification	CNN lr=0.0003 no class weights	0.866667	0.865500	0.867411	0.864510	0.872771	5	0.865500	0.0003	0.0001	
3	Cancer Classification	CNN lr=0.001 weight decay	0.869413	0.864926	0.868682	0.869864	0.861660	18	0.864926	0.0010	0.0001	

Figure E.1. Cancer CNN hyperparameter tuning results

The best tuned model used learning_rate = 0.0003 with class-weighted loss. It achieved a validation macro_f1 of 0.8727 at epoch 9. However, this did not improve on the original Improved CNN, which achieved a validation macro_f1 of 0.8764. This result shows that tuning the optimiser settings alone was not enough to reach the required 0.90 target. Although the tuned model produced reasonable results, it did not generalise better than the original CNN. This suggested that the model needed a stronger improvement approach, focused on changing how it was exposed to the training data rather than only adjusting optimiser settings.

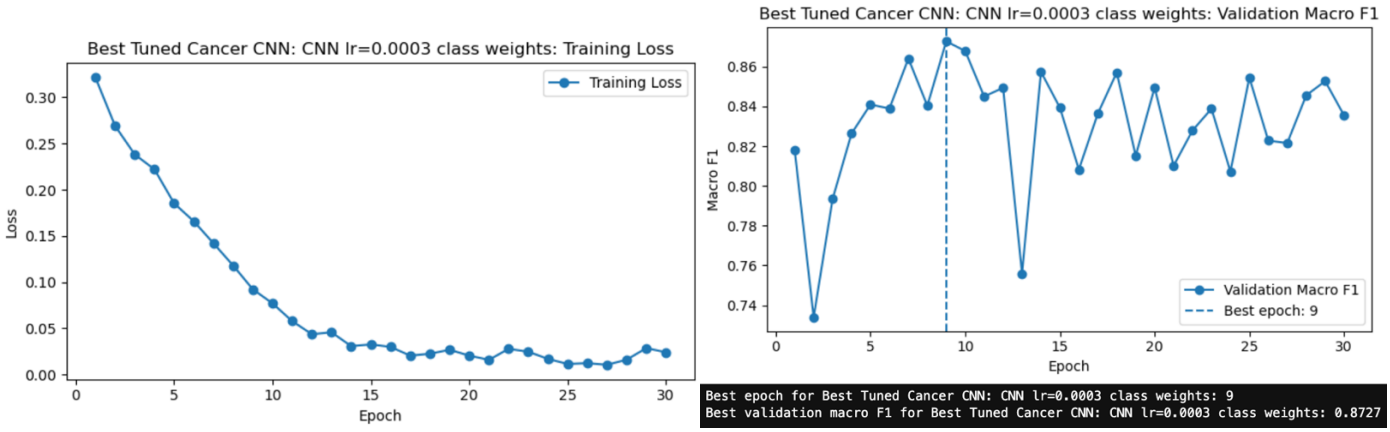

Figure E.1.2 Training loss and validation macro F1 for the best tuned cancer CNN.

E.2 Data augmentation

The next improvement strategy was data augmentation. Data augmentation was selected because the dataset consists of small 27 × 27 histology image patches, and the model needs to generalise to unseen patients. Instead of changing the model architecture again, data augmentation increases the variety of the training data by applying realistic transformations. Data augmentation was applied only to the training set. The validation and test sets were not augmented, as they must remain unchanged to provide a fair evaluation.

The augmentation pipeline used:

Augmentation technique	Setting	Purpose
Random horizontal flip	p = 0.5	Adds orientation variation
Random vertical flip	p = 0.5	Adds orientation variation
Random rotation	±15°	Reduces sensitivity to small rotations
Colour jitter	0.10 brightness, contrast, and saturation	Adds mild staining and lighting variation

These transformations are appropriate for histology patches because small changes in orientation or staining should not change whether a cell image is cancerous.

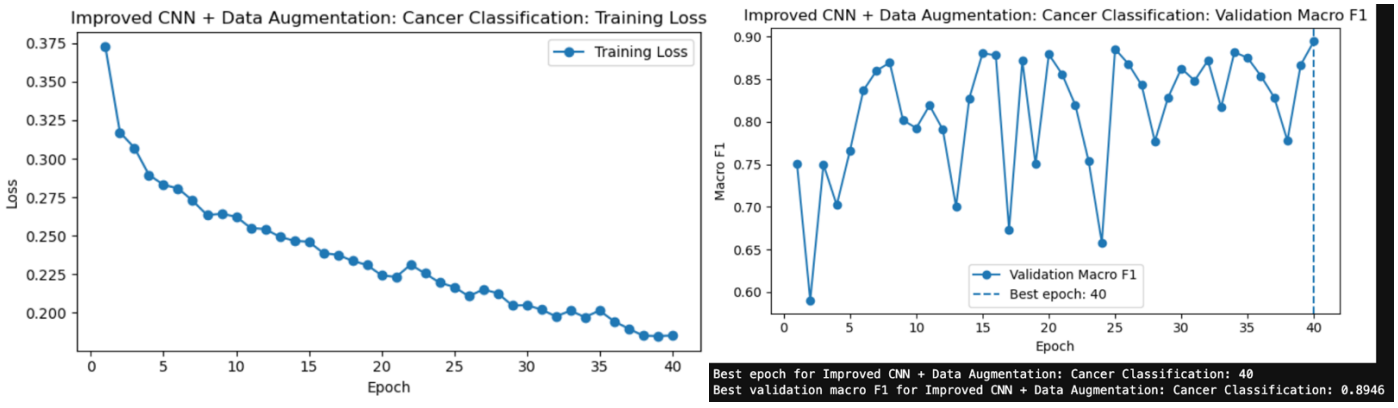

Figure E.2. Training loss and validation `macro_f1` for the cancer CNN with data augmentation

Data augmentation improved the cancer CNN validation macro_f1 from 0.8764 to 0.8946. This was a clear improvement over both the original CNN and the tuned CNN. The result suggests that the original model was limited partly by generalisation, rather than only by optimiser settings. However, the augmented CNN was still slightly below the required 0.90 target. This led to the final advanced step: decision threshold tuning.

E.3 Decision threshold tuning

Decision threshold tuning was applied after data augmentation. This technique was suitable because the cancer classification task is binary. The CNN outputs class probabilities, and these probabilities must be converted into class predictions. The default decision rule may not maximise macro_f1, especially when the classes are imbalanced. The

validation set was used to test different thresholds for predicting the cancerous class. The test set was not used during threshold selection.

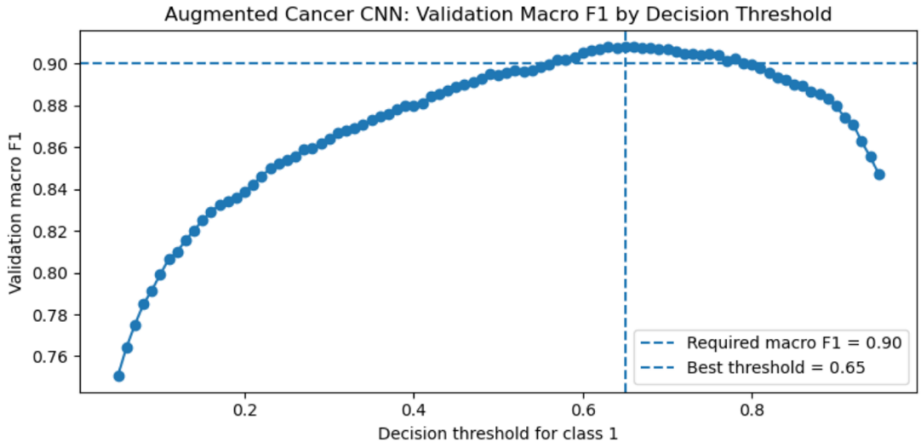

Figure E.3. Validation macro_f1 for the augmented cancer CNN across decision thresholds

The threshold tuning curve showed that the best validation result occurred at a threshold of 0.65. Using this threshold, the augmented CNN achieved a validation macro_f1 of 0.9079, exceeding the required 0.90 target. This indicates that the augmented CNN was already producing useful probability outputs, but the default decision rule was not optimal for this evaluation metric. Threshold tuning improved the balance between precision and recall across the two cancer classes.

```
================================================================================
Task: Cancer Classification Test Set
Model: Final Cancer Model: Improved CNN + Data Augmentation + Threshold Tuning
================================================================================
Accuracy:        0.8941
Macro F1:        0.8710
Weighted F1:     0.8967
Macro Precision: 0.8559
Macro Recall:    0.8928

Classification Report:
              precision    recall  f1-score   support

           0       0.96      0.90      0.93      2909
           1       0.75      0.89      0.82      1047

    accuracy                           0.89      3956
   macro avg       0.86      0.89      0.87      3956
weighted avg       0.90      0.89      0.90      3956
```

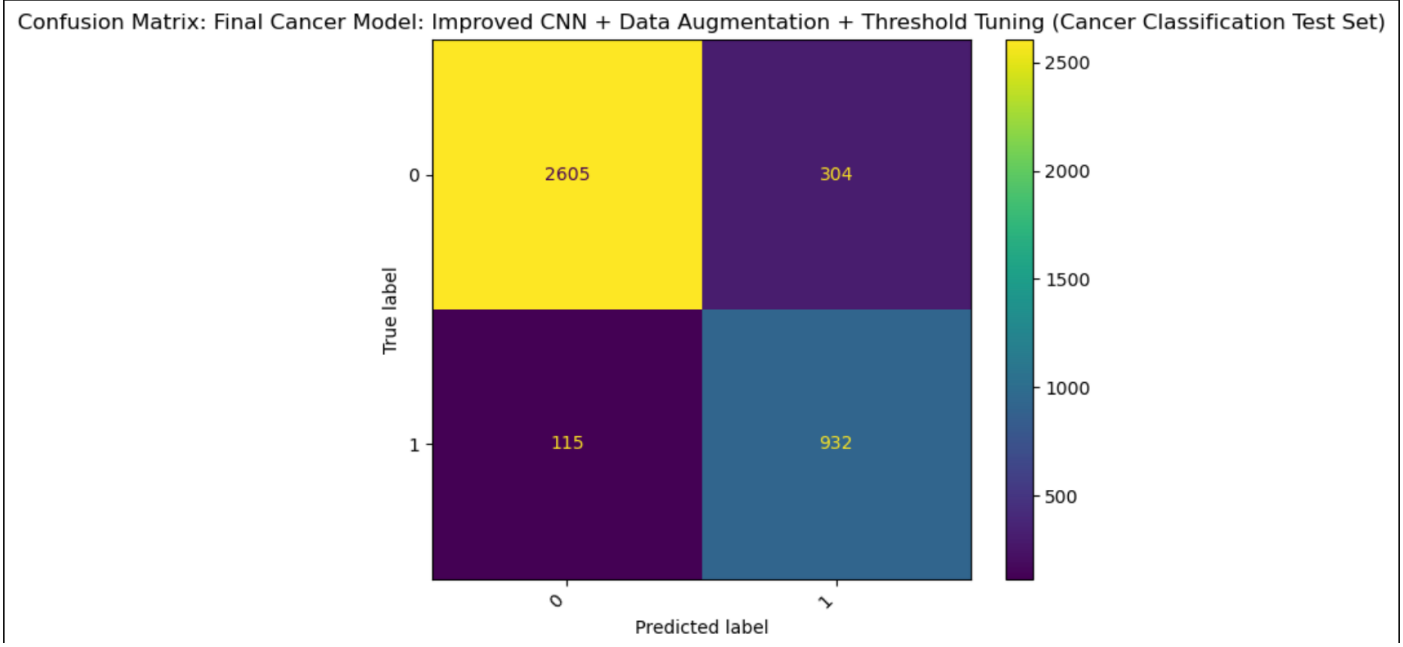

Figure E.3.2 Validation results and confusion matrix for the cancer CNN after data augmentation and threshold tuning

The final advanced cancer model correctly classified 2,131 non-cancerous images and 1,514 cancerous images. It made 177 false positive errors and 183 false negative errors. This relatively balanced error pattern supports the improved macro_f1 score.

Overall, the advanced technique results show a clear progression:

Model	Validation accuracy	Validation `macro_f1`
Original Improved CNN	0.8797	0.8764
Best tuned cancer CNN	0.8754	0.8727
Improved CNN + Data Augmentation	0.8961	0.8946
Improved CNN + Data Augmentation + Threshold Tuning	0.8961	0.9079

The best result was achieved by combining data augmentation with threshold tuning. Hyperparameter tuning was still useful because it showed that changing the optimiser settings alone was not sufficient. Data augmentation helped the model generalise more effectively, while threshold tuning improved the final decision rule for the binary cancer classification task.

F. Model Comparison

Model comparison was based on validation performance, with `macro_f1` used as the main selection metric. This metric was prioritised because both tasks contain class imbalance, so a suitable model should perform well across all classes rather than mainly favouring the majority class. Other metrics, including `accuracy`, `weighted_f1`, `macro_precision`, and `macro_recall`, were also considered, but `macro_f1` was the main metric used to compare model suitability.

	task	model	accuracy	macro_f1	weighted_f1	macro_precision	macro_recall	best_epoch	best_val_macro_f1	threshold
10	Cancer Classification	Improved CNN + Data Augmentation + Threshold T...	0.910112	0.907927	0.910091	0.908122	0.907736	NaN	NaN	0.65
9	Cancer Classification	Improved CNN + Data Augmentation	0.896130	0.894648	0.896554	0.892485	0.898959	40.0	0.894648	NaN
8	Cancer Classification	Improved CNN + Threshold Tuning	0.881898	0.878110	0.881388	0.881918	0.875378	NaN	NaN	0.54
6	Cancer Classification	Improved CNN	0.879650	0.876364	0.879439	0.877790	0.875145	5.0	0.876364	NaN
2	Cancer Classification	SVM RBF	0.859925	0.856775	0.860015	0.856332	0.857250	NaN	NaN	NaN
4	Cancer Classification	Random Forest	0.836954	0.827703	0.833794	0.846749	0.820706	NaN	NaN	NaN
0	Cancer Classification	Logistic Regression	0.773034	0.770972	0.774287	0.770570	0.776634	NaN	NaN	NaN
7	Cell-Type Classification	Improved CNN	0.686754	0.637979	0.694149	0.633793	0.651460	11.0	0.637979	NaN
3	Cell-Type Classification	SVM RBF	0.602606	0.505300	0.587570	0.566720	0.550925	NaN	NaN	NaN
5	Cell-Type Classification	Random Forest	0.574376	0.434521	0.515536	0.586220	0.516340	NaN	NaN	NaN
1	Cell-Type Classification	Logistic Regression	0.528773	0.432366	0.525408	0.441512	0.453209	NaN	NaN	NaN

Table F.1. Validation performance comparison across all supervised and advanced models

For the cancer classification task, validation performance generally improved as the models became more appropriate for image data. Logistic Regression achieved a validation macro_f1 of 0.7710, making it the weakest cancer model. This result was still useful because it provided a linear baseline. However, the lower score suggests that cancer classification cannot be solved effectively using only a simple linear decision boundary over flattened pixel values.

Random Forest Classifier improved the cancer validation macro_f1 to 0.8277, showing that non-linear feature interactions were useful. However, it still performed below SVM RBF and the CNN-based models. This suggests that tree-based learning on flattened pixel features was not the most effective approach for this image dataset.

SVM RBF achieved a stronger cancer validation macro_f1 of 0.8568. This was the best classical machine learning model for the cancer classification task. The result suggests that non-linear decision boundaries are important for this dataset. However, because SVM RBF still uses flattened image vectors, it does not directly learn local spatial patterns in the image.

The original Improved CNN achieved a cancer validation macro_f1 of 0.8764, outperforming all classical models. This supports the importance of preserving image structure in histology image classification. The CNN was able to learn local colour, texture, and spatial patterns that flattened-vector models could not represent as effectively.

The advanced CNN models produced the strongest cancer classification results. Data augmentation improved the cancer validation `macro_f1` from `0.8764` to `0.8946`, suggesting that the model generalised better when trained with augmented images. Threshold tuning further improved the score to 0.9079, which was the best validation result for the cancer task and exceeded the required 0.90 target.

For the cell-type classification task, the overall pattern was similar, but the task was more difficult. Logistic Regression and Random Forest Classifier achieved similar validation macro_f1 scores of 0.4324 and 0.4345. SVM RBF improved the result to 0.5053 but still remained below the CNN. The Improved CNN achieved the strongest cell-type validation result, with a validation macro_f1 of 0.6380. This exceeded the required 0.60 target. The lower cell-type score compared with the cancer score is expected because cell-type classification is a four-class problem involving visually similar

categories. The model must distinguish between epithelial, fibroblast, inflammatory, and others, which is more complex than separating cancerous and non-cancerous images. Overall, the comparison shows three main findings First, the classical machine learning models were useful as baselines, but they were limited because they used flattened pixel representations. Second, the CNN was the strongest model family for both tasks because it preserved image structure and learned local features. Third, achieving the best cancer classification performance required more than simply choosing the right model. Data augmentation and threshold tuning were also needed to improve generalisation and optimise the final binary decision rule.

Based on validation `macro_f1`, the strongest models were:

Task	Best model	Validation `macro_f1`
Cancer Classification	Improved CNN + Data Augmentation + Threshold Tuning	0.9079
Cell-Type Classification	Improved CNN	0.6380

These two models were selected for final test set evaluation.

G. Final Model Selection, Justification and Evaluation

The final models were selected using validation macro_f1, supported by accuracy, weighted_f1, macro_precision, macro_recall, and confusion matrix analysis. The validation set was used for model comparison, hyperparameter tuning, data augmentation decisions, and threshold selection. The test set was kept separate until the final evaluation stage. This was important because the test set provides the final estimate of model performance on unseen patients. Since the dataset was split by patientID, the test results provide a stricter and more realistic assessment of generalisation than a random image-level split.

G.1 Final selected models

For the cancer classification task, the final selected model was Improved CNN + Data Augmentation + Threshold Tuning. This model was selected because it achieved the strongest validation result for the cancer task. The original Improved CNN achieved a validation macro_f1 of 0.8764. Data augmentation improved this to 0.8946, and threshold tuning increased it further to 0.9079. This exceeded the required validation target of 0.90. The final threshold used for predicting the cancerous class was 0.65.

For the cell-type classification task, the final selected model was the Improved CNN. This model was selected because it achieved the strongest validation result for the cell-type task, with a validation macro_f1 of 0.6380. This exceeded the required validation target of 0.60. Further advanced techniques were prioritised for the cancer classification task because the initial cancer CNN had not yet met the required target, whereas the cell-type CNN had already exceeded its validation target.

G.2 Final test set performance

The final selected models were evaluated on the held-out test sets. These test sets were not used during training, validation, hyperparameter tuning, data augmentation decisions, or threshold selection.

```
final_cancer_model = aug_cnn_cancer_original_setting
final_cancer_threshold = best_aug_cancer_threshold

final_celltype_model = cnn_celltype

print("Final cancer model: Improved CNN + Data Augmentation + Threshold Tuning")
print("Final cancer threshold:", round(final_cancer_threshold, 2))

print("\nFinal cell-type model: Improved CNN")
```

```
Final cancer model: Improved CNN + Data Augmentation + Threshold Tuning
Final cancer threshold: 0.65

Final cell-type model: Improved CNN
```

Figure G.2. Final selected models for test set evaluation.

```
================================================================================
Task: Cancer Classification Test Set
Model: Final Cancer Model: Improved CNN + Data Augmentation + Threshold Tuning
================================================================================
Accuracy:        0.8941
Macro F1:        0.8710
Weighted F1:     0.8967
Macro Precision: 0.8559
Macro Recall:    0.8928

Classification Report:
              precision    recall  f1-score   support

           0       0.96      0.90      0.93      2909
           1       0.75      0.89      0.82      1047

    accuracy                           0.89      3956
   macro avg       0.86      0.89      0.87      3956
weighted avg       0.90      0.89      0.90      3956
```

Figure G.2.2 Cancer classification test set performance for the final selected model.

Task	Final model	Accuracy	macro_f1	Weighted f1	Macro precision	Macro recall	Threshold
Cancer Classification	Improved CNN + Data Augmentation + Threshold Tuning	0.8941	0.8710	0.8967	0.8559	0.8928	0.65
Cell-Type Classification	Improved CNN	0.6210	0.5537	0.6484	0.5719	0.5845	N/A

Table G.2.3 Final test set performance summary for the selected models

The cancer model achieved a test macro_f1 of 0.8710. This was lower than its validation macro_f1 of 0.9079, which indicates that the model did not generalise to the unseen-patient test set as strongly as it performed on the validation set. However, the result still demonstrates reasonable performance under a patient-level evaluation setting, where all test patients were unseen during model development.
The cell-type model achieved a test macro_f1 of 0.5537, which was also lower than its validation macro_f1 of 0.6380. This decrease suggests that cell-type classification remained more difficult when evaluated on unseen patients. This is consistent with the earlier EDA and validation results, where the cell-type task involved four visually similar classes and class imbalance.

G.3 Cancer classification test evaluation

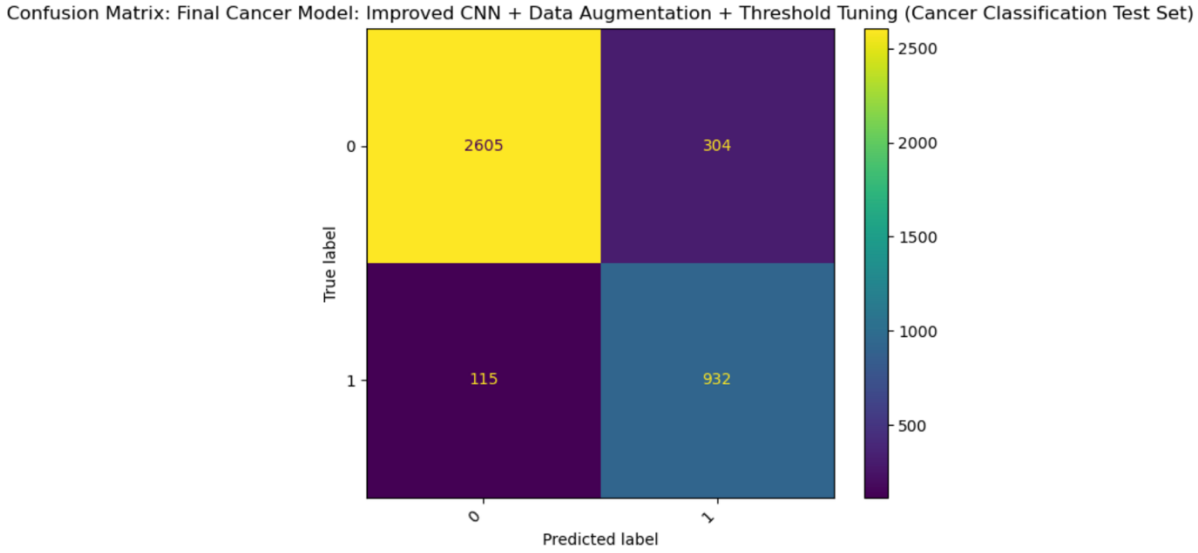

Figure G.3 Cancer classification test confusion matrix for the final selected model.

The cancer classification confusion matrix shows that the final model correctly classified 2,605 non-cancerous images and 932 cancerous images. It made 304 false positive errors and 115 false negative errors. The false negative count is particularly important in a cancer classification context because it represents cancerous images predicted as non-cancerous. In the test set, the model achieved a cancerous-class recall of 0.89, meaning it detected most cancerous samples. The non-cancerous class had higher precision and f1-score, but the cancerous class still achieved a reasonable f1-score of 0.82. The test macro_recall of 0.8928 suggests that the model maintained a reasonably balanced ability to identify both classes. However, the test macro_f1 of 0.8710 also shows that the model did not maintain the same level of performance observed during validation. This indicates that unseen-patient variation remained a significant challenge.

G.4 Cell-type classification test evaluation

```
======================================================================
Task: Cell-Type Classification Test Set
Model: Final Cell-Type Model: Improved CNN
======================================================================
Accuracy:        0.6210
Macro F1:        0.5537
Weighted F1:     0.6484
Macro Precision: 0.5719
Macro Recall:    0.5845

Classification Report:
              precision    recall  f1-score   support

  epithelial       0.95      0.71      0.81       925
  fibroblast       0.49      0.43      0.46       261
inflammatory       0.56      0.52      0.54       432
      others       0.29      0.68      0.41       221

    accuracy                           0.62      1839
   macro avg       0.57      0.58      0.55      1839
weighted avg       0.71      0.62      0.65      1839
```

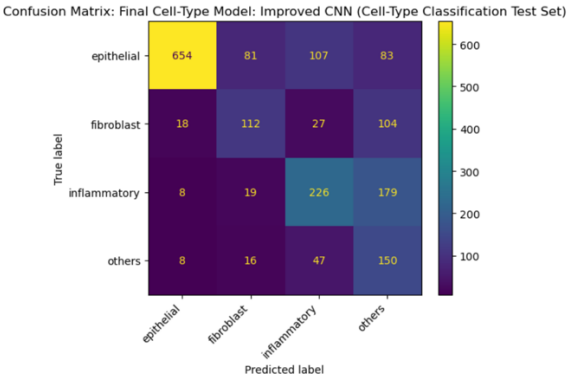

Table G.4 Cell-type classification test set performance and confusion matrix for the final selected model

The cell-type confusion matrix shows that epithelial was the easiest class for the model to identify, with 654 correctly classified epithelial images. This is consistent with the classification report, where epithelial achieved the strongest class-level f1-score of 0.81. The other three classes were more difficult. fibroblast achieved an f1-score of 0.46, inflammatory achieved 0.54, and others achieved 0.41. The model often confused fibroblast and inflammatory samples with others. This suggests that the visual boundaries between these cell categories are less distinct than the boundary between cancerous and non-cancerous images. The others class had relatively low precision but higher recall. This means the model predicted many images as others, including some images that belonged to other classes. This reduced the overall macro_precision and contributed to the lower test macro_f1. The cell-type result suggests that the CNN learned useful image features, but its performance was not fully consistent across unseen patients. Future work could therefore explore stronger regularisation, more targeted data augmentation, class-specific sampling, or additional model architectures.

	task	model	accuracy	macro_f1	weighted_f1	macro_precision	macro_recall	threshold
0	Cancer Classification	Improved CNN + Data Augmentation + Threshold T...	0.894085	0.871017	0.896691	0.855883	0.892830	0.65
1	Cell-Type Classification	Improved CNN	0.620990	0.553695	0.648366	0.571947	0.584507	NaN

Table G.4.2 Final test set performance summary for the selected models.

G.5 Validation and test performance comparison

The final test results were lower than the validation results for both tasks. This difference is important because it demonstrates the challenge of patient-level generalisation. For cancer classification, validation macro_f1 decreased from 0.9079 to 0.8710 on the test set. For cell-type classification, validation macro_f1 decreased from 0.6380 to 0.5537. This decrease does not mean that the workflow was inappropriate. Instead, it shows that the test set was more challenging and that patient-level splitting created a stricter evaluation setting. If a random image-level split had been used, the scores may have appeared higher, but they would have been less reliable because of possible patient leakage.

Overall, the final evaluation shows that the selected CNN-based models were the most appropriate models developed in this project. The cancer model performed reasonably well on unseen patients, although it did not fully maintain its validation-level performance. The cell-type model showed useful learning but struggled more clearly with patient-level variation and visually similar classes. These results highlight the importance of evaluating medical image classification models on unseen patients rather than relying only on validation performance.

H. Ethical Considerations and Professional Responsibilities

This project uses medical image data, so ethical responsibility is an important consideration, even though the work is conducted in an academic setting. The models developed in this report should be treated as experimental machine learning models, not clinical diagnostic tools. Their predictions may assist with analysis, but they should not replace the judgement of qualified medical professionals. One major ethical concern is the risk of incorrect predictions. In the cancer classification task, false negatives are especially serious because they mean cancerous images are predicted as non-cancerous. False positives are also important, as they may cause unnecessary concern or lead to further investigation. For this reason, the report considers confusion matrices, precision, recall, and macro_f1, rather than relying only on accuracy. Another professional responsibility is to ensure fair and realistic evaluation. A random image-level split could allow images from the same patient to appear in both training and testing sets, which may produce overly optimistic results. To reduce this risk, the data was split by `patientID`, so the test patients were not used during training or validation. This provides a stricter and more honest estimate of generalisation. The dataset also has limitations. The images are small, the class distributions are imbalanced, and the data may not represent all patient populations, hospitals, staining conditions, or imaging processes. Because of these limitations, the final models would require further validation on larger and more diverse external datasets before any real-world use. Professional responsibility also includes transparency. This report documents the preprocessing steps, model choices, validation results, advanced techniques, and final test performance. The test results were lower than the validation results, and this limitation is reported clearly

rather than hidden. This is important because machine learning reports should communicate both model strengths and weaknesses, particularly when the application area involves medical data.

I. Conclusion

This report developed and evaluated machine learning models for two histology image classification tasks: cancer classification using isCancerous and cell-type classification using cellTypeName. The workflow covered data inspection, EDA, patient-level splitting, preprocessing, classical machine learning baseline models, CNN modelling, hyperparameter tuning, data augmentation, threshold tuning, model comparison, and final test evaluation. The EDA identified several key challenges in the dataset, including class imbalance, low-resolution images, visually similar staining patterns, and patient-level variation. These findings supported the use of macro_f1 as the main evaluation metric and justified splitting the dataset by patientID to avoid patient leakage. Across the supervised models, the CNN-based approach performed best for both tasks. Classical models such as Logistic Regression, SVM RBF, and Random Forest Classifier provided useful baselines, but they were limited by their use of flattened pixel representations. The Improved CNN was more suitable for this image classification problem because it preserved image structure and could learn local colour and texture patterns. For cancer classification, the best validation result was achieved by Improved CNN + Data Augmentation + Threshold Tuning, with a validation macro_f1 of 0.9079. For cell-type classification, the selected model was the Improved CNN, which achieved a validation macro_f1 of 0.6380. Final test performance was lower for both tasks, showing that unseen-patient generalisation remained challenging. Overall, the project shows that CNN-based models are more appropriate for this histology image classification problem than classical flattened-feature models. However, the results also highlight that medical image classification requires careful validation, transparent reporting, and further testing on diverse unseen data before practical use.

J. References.

Fayek, H. (2026). Unsupervised learning: COSC2673/COSC2793 (Computational) Machine Learning [Lecture slides]. RMIT University.

Fayek, H. (2026). Other learning paradigms and practical considerations: COSC2673/COSC2793 (Computational) Machine Learning [Lecture slides]. RMIT University.

Pillow contributors. (2026). Pillow (PIL Fork) documentation. https://pillow.readthedocs.io/

PyTorch contributors. (2026). Torchvision documentation. https://docs.pytorch.org/vision/

RMIT University. (2026). Week 8 lab: Deep learning and neural network modelling [Jupyter notebook]. COSC2673 Computational Machine Learning. RMIT University.

RMIT University. (2026). Week 9 lab: Unsupervised learning [Jupyter notebook]. COSC2673 Computational Machine Learning. RMIT University.

RMIT University. (2026). Week 10 lab [Jupyter notebook]. COSC2673 Computational Machine Learning. RMIT University.

RMIT University. (2026). Week 11 lab [Jupyter notebook]. COSC2673 Computational Machine Learning. RMIT University.

RMIT University. (2026). Week 12 lab [Jupyter notebook]. COSC2673 Computational Machine Learning. RMIT University.

Zheng, X., & Fayek, H. (2026). Deep learning: COSC2673/COSC2793 (Computational) Machine Learning [Lecture slides]. RMIT University.